Contents

FitFolio backend architecture **1**
- Architecture . . . 1
- Modules . . . 1
- A module's files . . . 3
- How to read the class diagrams . . . 3
- Rules . . . 4
- Design patterns . . . 4
- How it runs . . . 4
- Migrations . . . 4

FitFolio backend architecture

Date	Version	Author	Notes
Nov 5, 2026	1.0	Davis Lenover	PoC version
Nov 7, 2026	1.1	Davis Lenover	Module health, users and wallet
Nov 8, 2026	1.2	Davis Lenover	Trading
Nov 8, 2026	1.3	Davis Lenover	One doc per module, access specifiers in the class diagrams

This is the high-level view. Each module is explained in its own doc, linked below. It covers only what the PoC needs, and grows as features are added.

Architecture

Client–server. The phone app is the client. It talks to one backend server over HTTPS, sending and receiving JSON (a REST API).

The server is a modular monolith on a small kernel. It is one FastAPI process with one PostgreSQL database, split into modules. - **Modules:** one per feature. Each is a folder in `src/backend/app/` and owns its own tables. - **Layers:** modules are stacked, and calls only go down. Trading calls market, but market never calls trading. - **Public interfaces:** other modules use a module only through its public files. - **Kernel:** `app/core/` holds what every module shares, and never imports a module. - **Plug-ins:** each module adds its routes to the app through its `plugin.py`, which is listed in `app/registry.py`. - **Adapter:** market gets prices through a provider class, so Finnhub can be swapped for another provider without changing other modules.

The server isn't split into separate services, because a buy must change the balance and the shares in one database transaction (FR_MktAtomicTrade). CI rejects any import that breaks these rules.

The rows inside "Modules" show the stacking. Trading uses market and wallet, activity uses wallet, and wallet uses users. Wallet and trading also use users to get the current user. Market's routes need no user.

The diagram's source is `figures/backend-poc.puml`. After changing a diagram, run this from the repository root to update its PNG:

```
plantuml -tpng docs/Design/SoftArchitecture/figures/backend-poc.puml
```

Modules

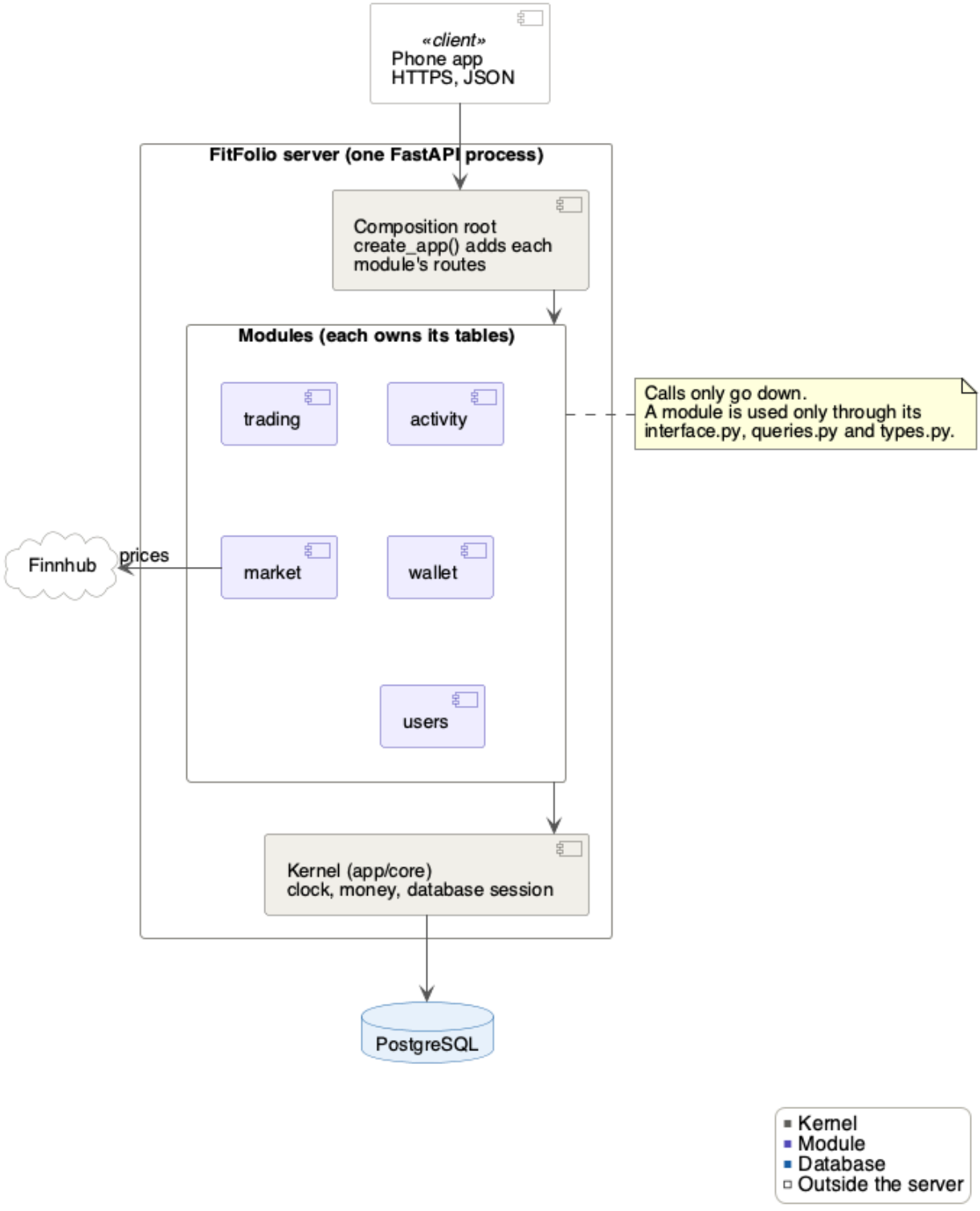


Figure 1: UML component diagram of the PoC backend

Module	What it does	Owns these tables	Routes	Issue
Kernel (`app/core/`)	What every module shares	none	`GET /v1/health`	#187
Users	Who is calling	`users`	none	#279
Wallet	Each portfolio's currency, and the one safe way to change it	`portfolios`, `balance_changes`	`GET /v1/me/balance`	#279, #189
Market	Stocks and their prices	`assets`, `prices`, `provider_requests`	`GET /v1/assets`, `GET /v1/market/usage`	#187
Trading	Buying shares, and what a portfolio is worth	`trades`, `holdings`	`POST /v1/me/trades`, `GET /v1/me/portfolio`	#188
Activity	Step rewards	`activity_daily_totals`	`PUT /v1/me/activity/daily-totals`	#190, not built yet

Each module's pull request adds or updates its doc in `modules/`, and its UML class diagram in `figures/`.

A module's files

File	What it holds	Other modules may import it
`interface.py`	Functions other modules may call	Yes
`queries.py`	Reads other modules may call. They never write	Yes
`types.py`	Data classes and errors	Yes
`service.py`	The rules. Never commits	No
`models.py`	The tables	No
`routes.py`	HTTP routes. Commits once	No
`plugin.py`	Adds the module to the app: its routes, background work and health check	No, only `app/registry.py`

Leave out any file a module doesn't need.

How to read the class diagrams

Each kind of box has one color in every diagram, shown in its legend. For example, an entry point is orange, a public file green, logic purple, a table or record blue and an error red. An arrow means "uses", and a hollow arrowhead means "is a".

Each member has an access specifier. In a box for a file, it follows the import rules below: - `+` **public:** other modules may use it, because it is in the module's `interface.py`, `queries.py` or `types.py`. A route is `+` too, since the phone uses it. - `~` **module:** only the module's own files may use it, such as everything in `service.py` and `models.py`. - `-` **private:** only its own file uses it. In Python its name starts with `_`.

In a box for a class, such as a record, a table, an error or a provider, `+` is a member that anyone holding one may use, and `-` is one whose name starts with `_`. Which file the class is in says who may import it.

Rules

1. Import another module only through its `interface.py`, `queries.py` or `types.py`.
2. Never import a module above yours in the diagram.
3. Never use another module's tables. A foreign key into them is fine, such as `trades.asset_id` to `assets`.
4. A route commits once per request. Code that answers no request commits once for each piece of work, in its own session, such as each of market's polls or one run of `seed_demo`. Nothing below them commits.
5. Money only moves through wallet. Lock the portfolio with `wallet.interface.lock_live_portfolio()` before calling `change_balance`, as wallet shows.
6. Take the time from `app/core/clock.py`, never from `datetime.now()`, so tests can set it.

CI checks rules 1 and 2. import-linter (`uv run lint-imports`) refuses an import of a module above, or of any module from `app/core/`. `test_public_files.py` refuses an import of another module's other files.

Design patterns

Plug-in kernel. The kernel (`app/core/` and `create_app()`) doesn't know which modules exist. Each module fills in a `Plugin` in its `plugin.py`, with its routes, any work that runs in the background, and a health check, and `app/registry.py` lists them. Adding a module never changes the kernel. See the kernel.

Strategy, adapter and factory. Market asks for prices through one interface, `MarketDataProvider`. `FinnhubProvider` adapts Finnhub's API to it, `FakeProvider` makes up prices, and `build_provider` picks one from `MARKET_PROVIDER`. Each provider gives its own request limit, so a new one changes no code outside `market/providers/`. See market.

Dependency injection. Code is handed what it needs instead of fetching it. Routes get a database session and the clock from the kernel, through `Depends(get_db)` and `Depends(get_clock)`, and the current user from users, through `Depends(get_current_user)`. The poller is given its provider, clock and session factory. Tests hand in stand-ins, such as a fixed time or a stub provider.

Read-only records. A module hands other modules frozen copies, such as market's `CurrentPrice` or wallet's `BalanceChangeReceipt`, never database rows. That way no module can change another module's tables by accident.

Lock token. Wallet's `change_balance` only takes the `LockedPortfolio` that `lock_live_portfolio` returns, so changing a balance without locking it first is hard to do by accident. See wallet.

Idempotency key. Each buy carries a `request_id` from the phone, so a retried request gets its first answer back instead of buying twice. See trading.

How it runs

When the app starts, the kernel adds each module's routes and starts each module's background work. For market, that is the poller thread, which saves a price for every stock once a minute while the market is open.

A request goes from the kernel to the module's route, which reads through the module's queries.

Migrations

There is still one chain, and 0001 to 0003 don't change. Each new migration is named after its module and numbered within it, such as `market_0002`. If `main` gets a new migration while you work, set your migration's `down_revision` to it before you merge, and keep its name. The backend README shows the commands.

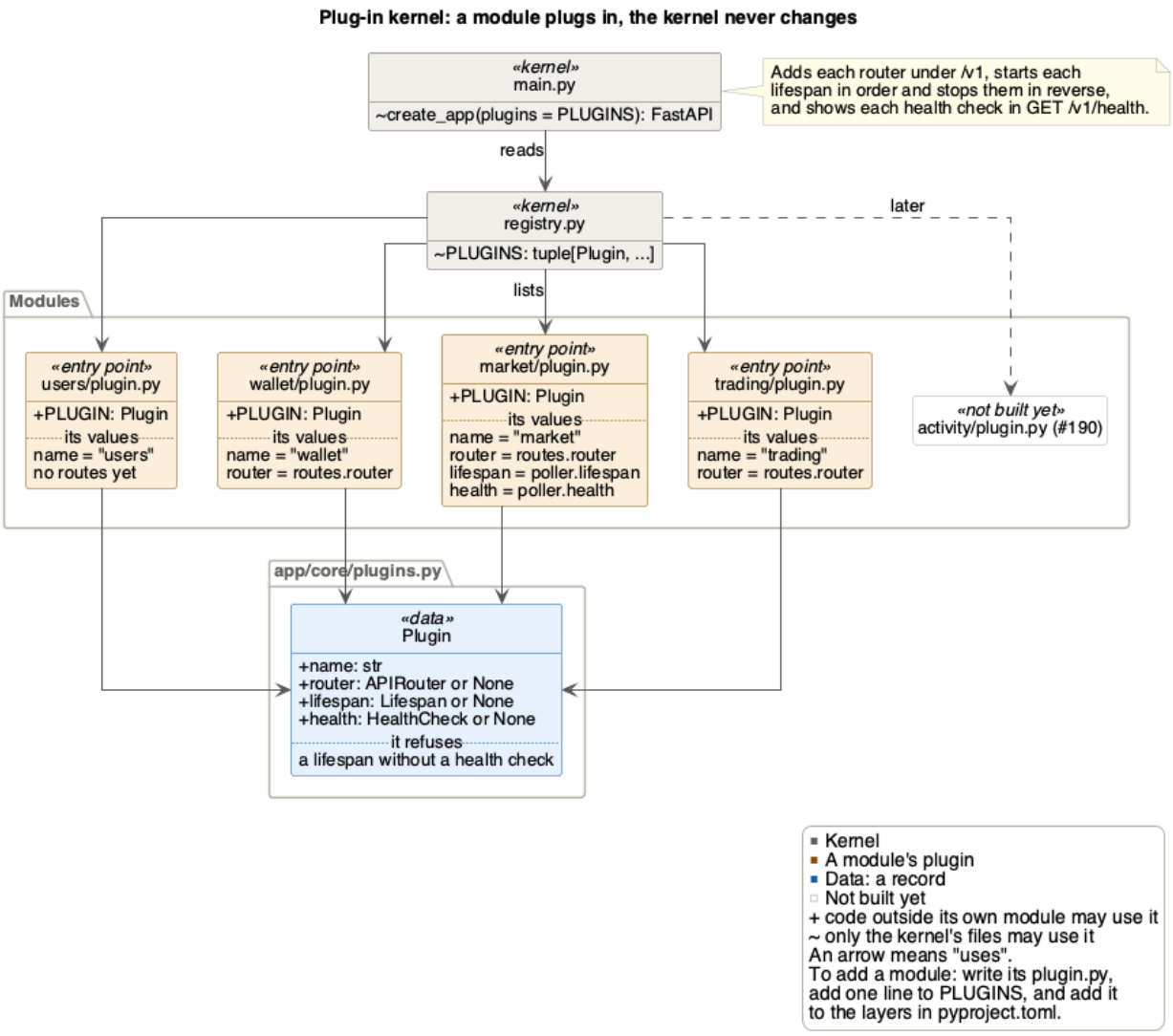


Figure 2: UML class diagram of the plug-in kernel

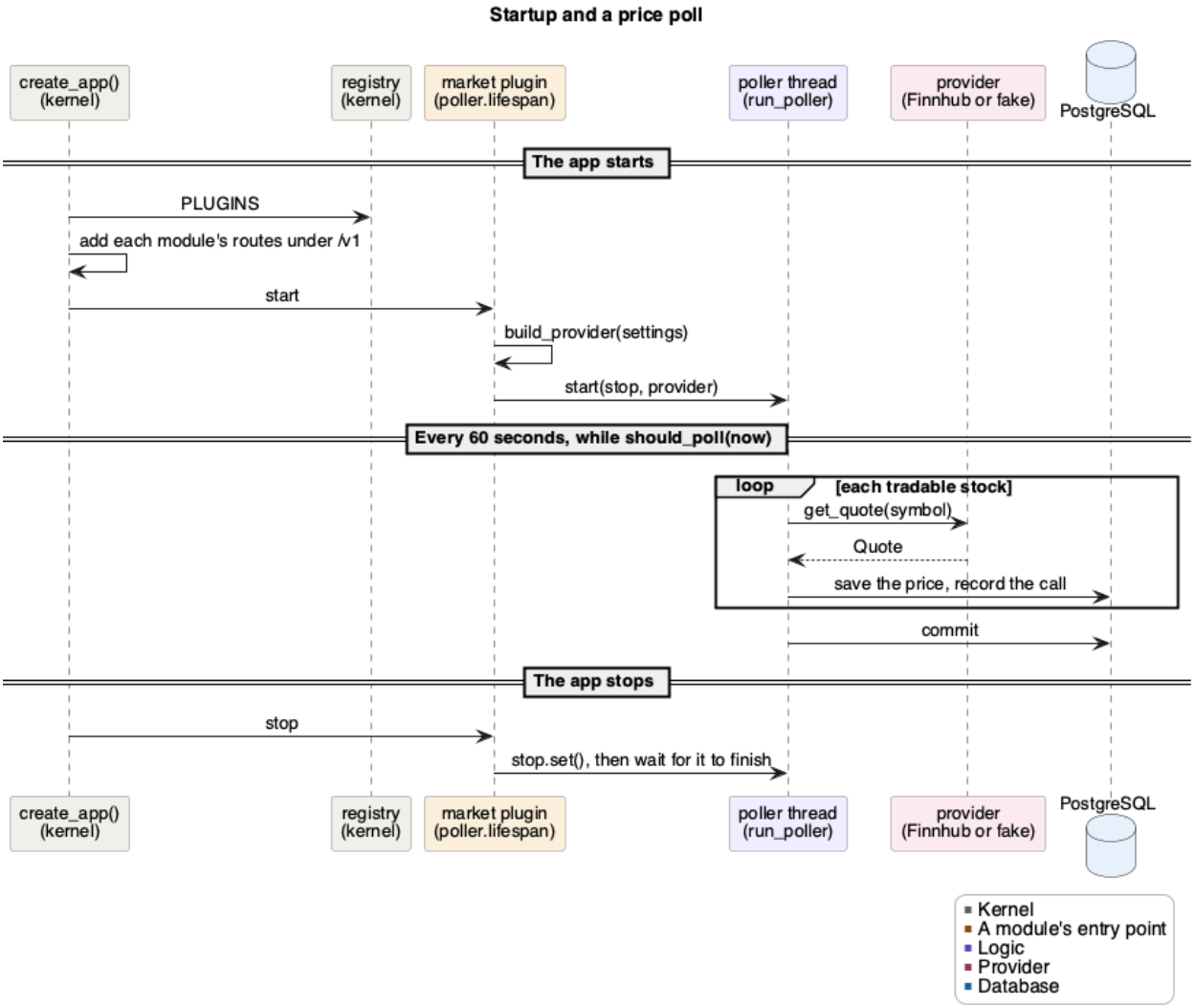


Figure 3: UML sequence diagram of startup and a price poll

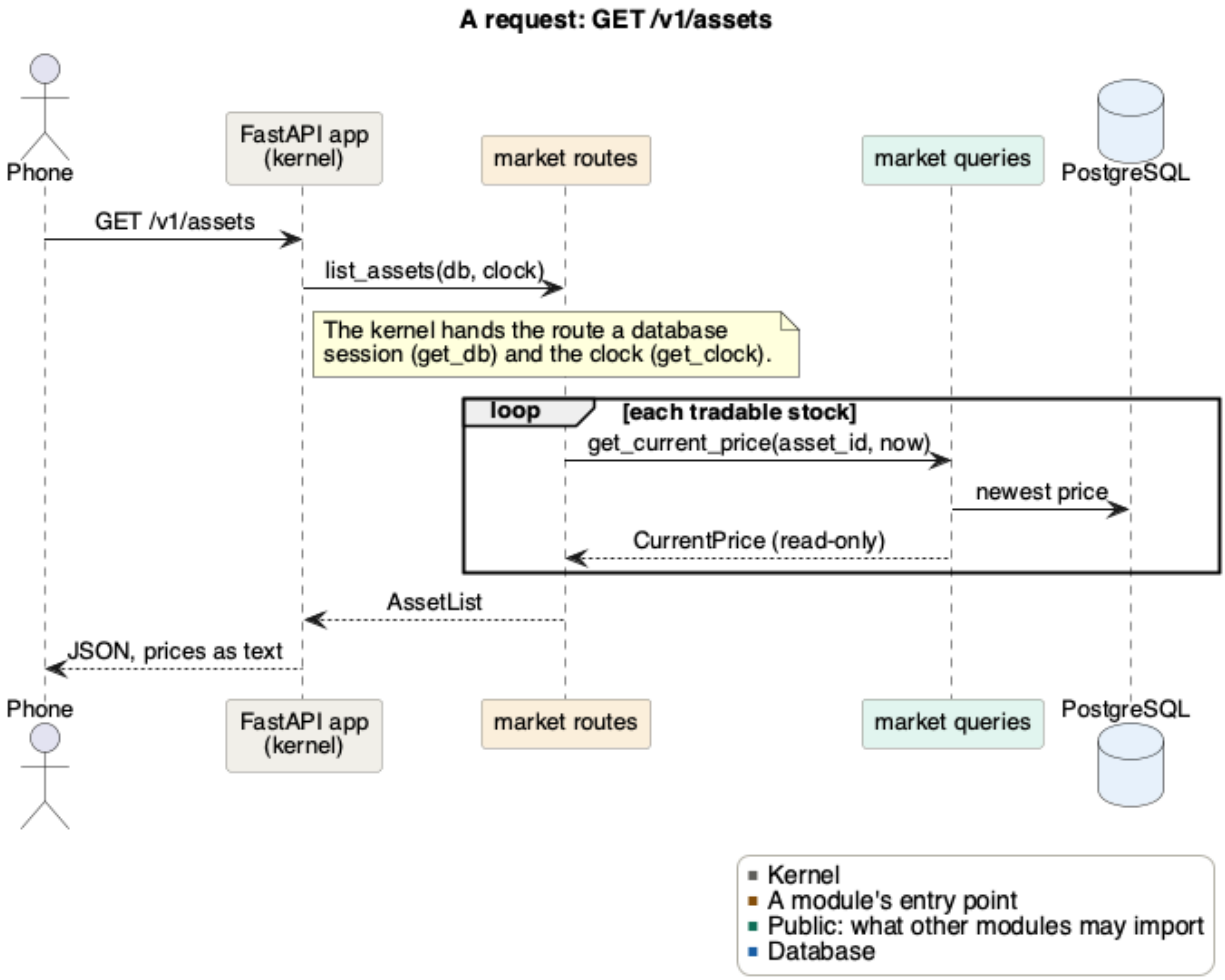


Figure 4: UML sequence diagram of a request for the asset list